核桃编程

L12-2　模块1：初识递推
模块2：递推的应用

1. 单词卡

Fibonacci	Hanoi
斐波那契	汉诺塔

2. 知识探索

（1）递推

①概念：通过已知的初始条件和递推公式来求解后续项的算法。

②步骤：

（1）定义初始条件：问题的初始条件，通常是初始项

（2）确定递推公式：找到问题中的规律，将规律使用公式表示，即为递推式。

（3）推导：利用递推公式，逐步推导出后续项。最终根据问题的项数u，得到问题的解。

（2）递推经典例题

斐波那契数列

```
a[1] = 1;
a[2] = 1;
for(int i = 3; i <= n; i++)
{
    a[i] = a[i-1]+a[i-2];
}
```

斐波那契数列，从第三个数字开始，每一个数字都等于前两个数字之和。
规律从第三个数字开始，先要对前两个数字做预处理，将其赋值为1。使用for循环推导后续数字，递推式a[i]=a[i-1]+a[i-2]。

爬楼梯问题

```
a[1] = 1;
a[2] = 2;
for(int i = 3; i <= n; i++)
{
    a[i] = a[i-1]+a[i-2];
}
```

对于n级阶梯，最后一步只能走1级或者2级，n级阶梯的走法数量也就等于最后一步走1级的走法数量加上最后一步走2级的走法数量，也就是n-1级阶梯的走法数量加n-2级阶梯的走法数量。
即a[i]=a[i-1]+a[i-2]

汉诺塔问题

```
a[1] = 1;
for (int i = 2; i <= n; i++)
{
    a[i] = 2 * a[i-1] + 1;
}
```

移动圆盘的过程分为三部分，第一部分，将1至n-1号圆盘移动到1号柱上，第二部分，将n号圆盘移动到3号柱上，第三部分，将1至n-1号圆盘移动到3号柱上。
因此，移动n个圆盘所需要的次数也就等于移动n-1个圆盘所需要的次数*2再加1。
递推公式：a[i] = 2 * a[i-1] + 1;

杨辉三角

杨辉三角是一种特殊的排列，它每一行最左边和最右边的元素都是1，其他元素的值等于上一行与其相邻的两个元素的和。
标准的杨辉三角形：

```
          1
        1   1
      1   2   1
    1   3   3   1
  1   4   6   4   1
```

左对齐的杨辉三角形：

```
1
1   1
1   2   1
1   3   3   1
1   4   6   4   1
```

递推公式：

$$f[i][j] = f[i-1][j-1] + f[i-1][j]$$

初始条件：

每一行的第一个和最后一个数为1，

即 $f[i][1] = f[i][i] = 1$。

```
int f[35][35], n;
int main()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
    {
        f[i][1] = f[i][i] = 1;
    }
    for (int i = 3; i <= n; i++)
    {
        for (int j = 2; j < n; j++)
        {
            f[i][j] = f[i-1][j-1] + f[i-1][j];
        }
    }
    for (int i = 1; i <= n; i++)
    {
        cout << f[n][i] << " ";
    }
    return 0;
}
```

初始化每一行第一个数和最后一个数为1

递推求解杨辉三角

错排问题

n个位置放置了**n**小球，如果改变小球的位置，使每个球都不在自己刚开始的位置上，这种排列方式为错位排列。

例如，**n=3**时，三个小球的初始排列为**a1 a2 a3**。

错位排列时，只有**a3 a1 a2**和 **a2 a3 a1**两种错位排列方案。

问题需要求解**n**个小球的错排方案数。

f[i]表示**i**个小球的错排方案数

f[1] = 0, f[2] = 1

递推式：**(i >= 3**时**)**

$$f[i] = (i-1) * (f[i-1] + f[i-2])$$

```
#include <iostream>
using namespace std;
int n, f[25];
int main()
{
    cin >> n;
    f[1] = 0;
    f[2] = 1;
    for (int i = 3; i <= n; i++)
    {
        f[i] = (i-1) * (f[i-1] + f[i-2]);
    }
    cout << f[n];
    return 0;
}
```

装弹问题--允许有炮口为空

问题描述：**m**枚相同的炮弹，装入**n**个炮口，允许有炮口为空，求所有装填的方案数。

f[i][j]表示**i**枚炮弹装入**j**个炮口的方案数。

f[0][0] = 1。

递推式：

当**i >= j**时

f[i][j] = f[i-j][j] + f[i][j-1]

否则

f[i][j] = f[i][i]

```
f[0][0] = 1;
for (int i = 0; i <= m; i++)
{
    for (int j = 1; j <= n; j++)
    {
        if (i >= j)
        {
            f[i][j] = f[i-j][j] + f[i][j-1];
        }
        else
        {
            f[i][j] = f[i][i];
        }
    }
}
cout << f[m][n];
```